

Measuring a retrieval-augmented system

This document was written for the PaperMind project as a sample document for the evaluation fixture. It is released under CC0 1.0 (public domain): you may copy, modify, and redistribute it without permission or attribution. The measurements quoted in it are an example run made for the demo; they are illustrative numbers, not a published result.

Why a ground-truth set exists

A retrieval system cannot be judged by reading its answers. Two systems can produce the same answer for the question you happened to try, and differ completely on the next one. Measuring retrieval means fixing a set of questions in advance, deciding which passages answer each of them, and then checking what the system actually put in front of the reader.

That set is the only part of an evaluation that cannot be generated. Everything downstream of it, the metrics, the report, the comparison between two configurations, is arithmetic over a file a person wrote and checked.

The four retrieval metrics

Hit rate at k asks whether any of the passages that answer a question appeared in the first k results. It is one number per question, either one or zero, and the mean over the set is the share of questions the system can reach at all.

Recall at k asks what share of the passages a question needs appeared in the first k results. A question whose answer spans two sections has two gold passages, and recall is the metric that notices one of them is missing where hit rate is satisfied by the other.

Mean reciprocal rank gives credit for finding a gold passage early. A system that puts the right passage third is not as useful as one that puts it first, even though both score a hit, and reciprocal rank is the number that says so: one over the rank of the first gold passage, averaged over the set.

Normalized discounted cumulative gain is the same idea extended to a ranked list of several gold passages. It rewards a system for putting all of them near the top, in the order a reader would want them.

A measured comparison

Table 1 is the retrieval side of one example run over a tuning set of cases like this one, with k set to five.

Table 1. Retrieval quality by configuration, tuning cases, k = 5.

Configuration	Hit rate	Recall	MRR	nDCG	p95
Dense only	0.71	0.62	0.58	0.55	41 ms
Sparse only	0.66	0.70	0.61	0.60	12 ms
Hybrid	0.88	0.84	0.79	0.81	58 ms
Hybrid + rerank	0.94	0.93	0.88	0.90	96 ms

The two single-representation rows are the interesting pair. Sparse retrieval finds more of the gold passages on average than dense retrieval does, because a question that reuses the document's own wording is easy to match lexically, but it misses more questions outright, and the passages it does find sit further down the list.

Reranking costs most of the latency the hybrid row already spent. It is worth that only when the reader is going to read the answer rather than skim a heading.

Answer metrics

Retrieval metrics say nothing about the answer, because the retrieved passages are only ever going to be passages.

Correctness asks whether the answer says the thing the case's reference answer says. It is judged by a model reading the question, the reference answer, and the retrieved context, and it is reported as correct, incorrect, or unknown.

Faithfulness asks a narrower question: is every claim in the answer supported by the context that was supplied. An answer can be true and still unfaithful, and an unfaithful answer is the one that cannot be checked by a reader at all, because it points at a passage that does not say what the sentence claims.

Citation precision is the share of an answer's citations that support the sentence they sit on. Citation recall is the share of an answer's sentences that carry a citation at all. A reader who has to guess which passage backs a sentence is doing the retrieval system's job.

Abstention accuracy has two halves and both are errors. Abstaining on a question the evidence answers wastes the reader's time. Answering a question the evidence does not reach is worse, because the answer was invented.

Splitting the cases before anything is tuned

A set of cases used to choose a configuration and a set of cases used to report a result cannot be the same set. The moment they are, the reported number describes the cases that were fitted to, and it goes up every time somebody tries a new setting.

So the cases are split once, before any configuration is chosen. The tuning split is the one to look at while changing chunk size, overlap, candidate depth, or whether the reranker runs. The reported split is read once, at the end, and quoted. A configuration that only wins on the tuning split has been fitted to fourteen questions, and a reader should be told that rather than shown the number.

Latency and cost

The first question of a run pays for loading whatever loads lazily, so its seconds are reported apart from the rest. Averaging the cold start into the steady state describes a cost the reader never pays twice, and it makes a slow system look slower than it is. Report the pair.

Token counts come from the provider where the provider reports them, and from the application's own tokenizer where it does not. The cost estimate then has to travel with the prices it used, because a dollar figure without the price list behind it cannot be checked.

What a set this size cannot tell you

Forty to sixty reviewed cases can tell you that a change moved retrieval in a direction, that a regression exists, and that a specific question fails. It cannot tell you the system's accuracy, and a report that quotes a hit rate from sixty cases as though it were a rate for the whole corpus is overstating it.

Report the size, the origin, and the licensing of the set next to the number, and say which questions dragged an average down rather than only that it fell.